VISUAL RECOGNITION

Mini Project – 1

Multi-Object Apparel Detection and Instance Segmentation

Submitted By

IMT2023057 – K.Jitin Venkata Sai
IMT2023074 – Rishik Patha
IMT2023105 – Jupalli Prabhas
IMT2023108 – Mopuri Abhiram

GitHub Link

https://github.com/jitin1975/VR_MiniProject1

HuggingFace Link

https://huggingface.co/Ghost0509/VrRepoSubmission

Course: Visual Recognition

Date: March 31, 2026

Contents

1 Problem Statement **2**

2 Dataset Description **2**
2.1 Raw Dataset Statistics . . . 2
2.2 Category Frequency Analysis . . . 3

3 Data Preprocessing **3**
3.1 Dataset Pruning . . . 3
3.2 Choosing the Training Data Size . . . 3
3.3 Handling Class Imbalance . . . 4
3.4 Annotation Parsing (Task 2) . . . 4

4 Classification **5**
4.1 Problem Definition . . . 5
4.2 Dataset Curation and Analysis . . . 6
4.3 Preprocessing Pipeline . . . 8
4.4 Model Architectures . . . 10
4.5 Multi-Label Adaptation . . . 12
4.6 Training Methodology . . . 13
4.7 Experimental Details . . . 14
4.8 Results . . . 16
4.9 Comparative Analysis . . . 18
4.10 Discussion and Insights . . . 19
4.11 Conclusion . . . 21

5 Detection and Segmentation **22**
5.1 Problem Definition . . . 22
5.2 Dataset Preparation . . . 23
5.3 Preprocessing Pipeline . . . 25
5.4 Model Architectures and Training Strategies . . . 28
5.4.1 U-Net (Semantic Segmentation) . . . 28
5.4.2 YOLO (Detection and Segmentation) . . . 37
5.4.3 Mask R-CNN (Instance Segmentation) . . . 44

6 Final Comparative Analysis **53**
6.1 Classification Comparison . . . 53
6.2 Detection and Segmentation Comparison . . . 55
6.3 Overall Insights . . . 56

1 Problem Statement

The rapid growth of fashion e-commerce has created an urgent need for automated visual recognition systems capable of accurately identifying and localising clothing items in images. This project develops a comprehensive system addressing three core tasks on the DeepFashion2 dataset:

- **Task 1: Multi-Label Classification:** Given an input image, identify all clothing categories present using CNN-based models. No spatial localisation is required.
- **Task 2: Detection and Segmentation:** Given an input image, detect and classify all clothing item instances, predict their bounding boxes, and generate pixel-level segmentation masks.

The five most frequent clothing categories were selected from the 13 available in DeepFashion2:

- Short Sleeve Top (cat. 1)
- Trousers (cat. 8)
- Shorts (cat. 7)
- Long Sleeve Top (cat. 2)
- Skirt (cat. 9)

Each model was trained under two strategies:

- From scratch (random initialisation)
- Transfer learning (pretrained weights)

2 Dataset Description

DeepFashion2 is a large-scale, person-centric fashion dataset providing item-level instance annotations. Each image is accompanied by a JSON file containing:

- Bounding boxes
- Polygon segmentation masks
- Category IDs
- Occlusion levels
- Viewpoints
- Scale information

A single image can contain multiple clothing items, making it suitable for multi-instance detection and segmentation.

2.1 Raw Dataset Statistics

Split	Total Images	Images Retained (Top-5)	Notes
Train	191,961	144,174	∼75% retention rate
Validation	32,153	23,741	∼74% retention rate
Test	62,629	62,629	No annotations, inference only

Table 1: Dataset statistics after pruning

2.2 Category Frequency Analysis

A frequency analysis of all 13 categories in the training split was performed to select the top 5.

Category ID	Category Name	Frequency (Train)	Mapped Index	Selected
1	Short Sleeve Top	71,645	0	Yes
8	Trousers	55,387	1	Yes
7	Shorts	36,616	2	Yes
2	Long Sleeve Top	36,064	3	Yes
9	Skirt	30,835	4	Yes
12	Vest Dress	17,949	–	No
10	Short Sleeve Dress	17,211	–	No
5	Vest	16,095	–	No
4	Long Sleeve Outwear	13,457	–	No
Others	–	$<$ 8,000 each	–	No

Table 2: Category frequency distribution

The top 5 categories account for approximately 230,547 of the 312,331 total annotations (73.8%). Even within the top 5, there is a 2.3× imbalance between Short Sleeve Top and Skirt.

3 Data Preprocessing

3.1 Dataset Pruning

The raw DeepFashion2 dataset was pruned using a dedicated script. The pipeline scanned all 191,961 training annotation files to count per-category frequencies, identified the top-5 category IDs [1, 8, 7, 2, 9], and retained only images containing at least one item from these categories.

For retained images:

- Annotations for non-top-5 items were discarded
- Each annotation was saved in a minimal JSON format:

```
{items: [{category_id, bbox, segmentation}]}
```

Images were copied into a new directory to ensure portability.

3.2 Choosing the Training Data Size

Task 1 (Classification):

- Full dataset (144,174 images) used
- Batch size: 32 on Kaggle GPU
- Provides better diversity for classification

Task 2 (Detection and Segmentation):

- 20,000 training images
- 5,000 validation images

Reason:

- Mask R-CNN at full scale ∼660 GPU-hours
- Reduced dataset ∼11 hours
- Balanced subset ensures equal class representation

3.3 Handling Class Imbalance

Three strategies were used:

1. BCEWithLogitsLoss (Task 1)

$$pos_weight[i] = \frac{N - P_i}{P_i}$$

Category	Positive Count	pos_weight
Short Sleeve Top	70,586	1.04
Trousers	54,969	1.62
Shorts	36,332	2.97
Long Sleeve Top	35,751	3.03
Skirt	30,625	3.71

Table 3: Class imbalance weights

2. WeightedRandomSampler

Each sample weight was computed based on class presence to ensure rare categories are sampled more frequently.

3. Weighted CrossEntropy (Task 2)

Weights used:

$$[0.022, 0.698, 0.903, 1.367, 1.387, 1.623]$$

3.4 Annotation Parsing (Task 2)

For each image:

- Extract bounding boxes $[x1, y1, x2, y2]$
- Map class labels
- Convert polygons to masks using OpenCV

Model-specific usage:

- Mask R-CNN → Instance masks
- U-Net → Multi-class semantic mask
- YOLO → Normalized polygon coordinates

4 Classification

4.1 Problem Definition

The classification component of this project is formulated as a **multi-label image classification problem**, where each input image may contain multiple clothing categories simultaneously. Unlike conventional single-label classification tasks, where each image is assigned exactly one class, this problem requires the model to identify all relevant categories present in the image.

Formally, let I denote an input image. The objective is to learn a mapping:

$$f : I \to y$$

where:

$$y \in \{0, 1\}^C$$

is a binary label vector and $C = 5$ represents the number of selected clothing categories. Each element y_i indicates the presence (1) or absence (0) of the i-th category in the image.

This formulation is particularly relevant in real-world fashion scenarios, where individuals typically wear multiple garments simultaneously. For example, a single image may contain a *top*, *trousers*, and *outerwear*, all of which must be correctly identified.

Key Insight

Multi-label classification better reflects real-world scenarios where multiple objects or categories coexist within a single image, making it more complex yet more practical than traditional single-label classification.

Multi-Label vs Multi-Class Classification

It is important to distinguish between multi-label and multi-class classification:

- **Multi-class classification**: Each image belongs to exactly one class.
- **Multi-label classification**: Each image may belong to multiple classes simultaneously.

The DeepFashion2 dataset inherently supports a multi-label setting because each image contains multiple annotated clothing items.

Key Insight

The independence of labels in multi-label classification requires models to predict each category separately, increasing the complexity of the learning process.

Challenges in Multi-Label Classification

Multi-label classification introduces several challenges:

- **Label Co-occurrence**: Certain classes frequently appear together (e.g., tops and trousers), leading to correlation between labels.
- **Class Imbalance**: Some clothing categories appear significantly more often than others.
- **Visual Similarity**: Categories such as tops and dresses may have overlapping visual features.
- **Occlusion**: Clothing items may be partially hidden by other objects or persons.

Key Insight

Handling label correlation, class imbalance, and visual ambiguity is critical for achieving robust performance in multi-label classification tasks.

These challenges necessitate robust feature extraction and careful training strategies.

4.2 Dataset Curation and Analysis

The DeepFashion2 dataset was utilized for the classification task. This dataset provides detailed annotations for multiple clothing items within each image, including category labels, bounding boxes, and segmentation masks.

Dataset Characteristics

Each image in the dataset contains:

- Multiple clothing items
- Instance-level annotations
- Diverse conditions such as varying viewpoints, occlusions, and lighting

These characteristics make the dataset suitable for real-world visual recognition tasks but also introduce significant complexity.

Key Insight

The presence of multiple objects and real-world variations makes DeepFashion2 a challenging yet highly representative dataset for multi-label classification.

Category Selection

To ensure computational feasibility and sufficient representation per class, the dataset was pruned to retain the **top 5 most frequent clothing categories**.

Motivation:

- Reduces class sparsity
- Ensures balanced learning
- Enables stable convergence

Key Insight

Including rare classes with limited samples leads to poor generalization and unstable training, making category selection a critical step.

Dataset Size Selection

A subset of the dataset was selected instead of using the full dataset.

Rationale:

- Training on the entire dataset was computationally expensive
- Marginal performance gains diminished beyond a certain dataset size
- Efficient experimentation required a manageable dataset size

Key Insight

A carefully selected subset can achieve comparable performance while significantly reducing computational cost and training time.

Multi-Label Target Construction

The original dataset provides instance-level annotations. These were aggregated to form image-level labels.

For each image:

1. Extract all clothing instances
2. Collect their category IDs
3. Construct a binary vector representing presence/absence

Example:

- Categories present: Top, Trousers
- Label vector: [1, 0, 1, 0, 0]

Key Insight

Converting instance-level annotations to image-level labels simplifies the classification task while preserving multi-label information.

Train-Validation Split

The dataset was divided into training and validation sets using a standard split.

Considerations:

- Maintain class distribution across splits
- Avoid data leakage

- Ensure fair evaluation

Key Insight

Maintaining consistent class distribution across splits is essential for reliable and unbiased model evaluation.

Class Imbalance Analysis

An analysis of category distribution revealed imbalance across classes.

Impact:

- Models tend to favor dominant classes
- Minority classes may be underrepresented in predictions

Mitigation Strategies:

- Data augmentation
- Balanced sampling (implicitly handled through dataset pruning)

Key Insight

Class imbalance directly affects recall for minority classes, making it a critical factor in multi-label classification performance.

4.3 Preprocessing Pipeline

Preprocessing plays a crucial role in improving model performance by ensuring consistent input representation and enhancing generalization across diverse real-world conditions.

Image Resizing

All images were resized to a fixed resolution (e.g., 224×224 pixels) to ensure compatibility with CNN architectures.

Reason:

- CNNs require fixed input dimensions
- Reduces computational complexity
- Enables efficient batch processing

Normalization

Images were normalized using standard ImageNet statistics.

Purpose:

- Stabilizes training and improves convergence

- Ensures consistent feature scaling
- Aligns input distribution with pretrained model expectations

Data Augmentation

To improve generalization, several augmentation techniques were applied:

Techniques Used:

- **Horizontal Flipping**: Simulates mirror variations and improves orientation invariance
- **Rotation**: Handles viewpoint changes and minor pose variations
- **Brightness Adjustment**: Accounts for lighting variations in real-world images

Impact:

- Reduces overfitting by exposing the model to diverse samples
- Improves robustness to unseen conditions
- Enhances effective dataset size without additional data collection

Key Insight

Data augmentation is one of the most effective techniques to improve generalization, especially when training data is limited or imbalanced.

Why Preprocessing Matters

Without preprocessing:

- Models may overfit to training data
- Performance degrades significantly on unseen data
- Training becomes unstable and less efficient

Pipeline Summary

The preprocessing pipeline can be summarized as:

1. Load image
2. Resize to fixed resolution
3. Normalize pixel values
4. Apply random augmentations

Key Insight

Preprocessing acts as a foundation for model performance by ensuring that the network learns meaningful and generalizable features.

4.4 Model Architectures

To evaluate the effectiveness of different convolutional neural network (CNN) architectures for multi-label clothing classification, three models were considered:

- ResNet-50
- EfficientNet
- MobileNetV3

These models were selected to represent diverse architectural design philosophies, including deep residual learning, compound scaling, and lightweight efficiency-oriented networks.

Key Insight

Selecting architectures with different design principles enables a comprehensive comparison of accuracy, efficiency, and scalability.

ResNet-50

ResNet-50 is a deep convolutional neural network that introduces **residual connections** to address the vanishing gradient problem in deep networks.

Key Idea: Instead of learning a direct mapping $H(x)$, the network learns a residual function:

$$F(x) = H(x) - x$$

Thus, the original mapping becomes:

$$H(x) = F(x) + x$$

Advantages:

- Enables training of very deep networks
- Improves gradient flow through skip connections
- Captures rich hierarchical feature representations

Relevance to Task:

ResNet-50 is particularly effective in capturing complex visual patterns such as textures, shapes, and layered clothing structures, which are essential for multi-label classification.

EfficientNet

EfficientNet introduces a **compound scaling method** that uniformly scales network depth, width, and resolution.

Scaling Principle:

- Depth: number of layers
- Width: number of channels
- Resolution: input image size

Instead of scaling these dimensions independently, EfficientNet uses a principled approach to balance them for optimal performance.

Advantages:

- High accuracy with fewer parameters
- Better efficiency compared to traditional CNN architectures

Limitation Observed:

When trained from scratch, EfficientNet may underperform due to insufficient data and lack of pretrained initialization.

MobileNetV3

MobileNetV3 is a lightweight CNN architecture designed for mobile and embedded applications.

Key Components:

- Depthwise separable convolutions
- Squeeze-and-Excitation (SE) blocks
- Hard-Swish activation

Advantages:

- Low computational cost
- Fast inference speed
- Suitable for real-time and resource-constrained environments

Trade-off:

MobileNetV3 sacrifices some accuracy in exchange for computational efficiency.

Key Insight

Lightweight architectures such as MobileNetV3 are ideal for deployment scenarios, while deeper models like ResNet-50 are preferred for achieving maximum accuracy.

Architectural Comparison

- **ResNet-50**: Highest accuracy, higher computational cost
- **EfficientNet**: Balanced trade-off between performance and efficiency
- **MobileNetV3**: High efficiency with slightly reduced accuracy

Key Insight

The choice of architecture depends on the application requirements, balancing accuracy, computational resources, and deployment constraints.

4.5 Multi-Label Adaptation

Standard CNN architectures are designed for single-label classification using a softmax activation function. However, this task requires multi-label prediction.

Why Softmax is Not Suitable

Softmax assumes that:

- Only one class is correct
- Probabilities sum to 1

This assumption is invalid for multi-label problems where multiple classes may coexist.

Sigmoid Activation

To address this, the final layer of each model was modified to use a **sigmoid activation function**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

This allows independent probability estimation for each class.

Interpretation: Each output node predicts the probability of a specific class independently.

Loss Function: Binary Cross Entropy

The models were trained using Binary Cross Entropy (BCE) loss:

$$\mathcal{L} = -\sum_{i=1}^{C} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Advantages:

- Suitable for multi-label classification
- Handles independent class predictions

Thresholding Strategy

Predicted probabilities were converted into binary labels using a threshold.

Different thresholds were evaluated (0.3, 0.4, 0.5, 0.6).

Observation:

A threshold of 0.4 achieved the best macro F1-score, indicating an optimal balance between precision and recall.

Importance of Threshold Selection

- Lower threshold → higher recall, lower precision
- Higher threshold → higher precision, lower recall

Thus, threshold selection is critical in multi-label settings.

4.6 Training Methodology

Two distinct training strategies were employed to evaluate model performance:

- Training from scratch
- Transfer learning

Training from Scratch

In this approach, all model parameters were randomly initialized and trained end-to-end.

Advantages:

- Full control over learned feature representations
- No dependency on pretrained datasets

Limitations:

- Requires large amounts of data
- Slow convergence
- Lower performance during early training stages

Transfer Learning

In transfer learning, models were initialized with pretrained weights (e.g., ImageNet) and fine-tuned for the target task.

Procedure:

1. Load pretrained weights
2. Freeze early layers to retain generic features
3. Fine-tune later layers for task-specific learning

Advantages:

- Faster convergence
- Improved generalization
- Higher overall performance

Key Insight

Pretrained models capture generic visual features such as edges, textures, and shapes, which can be effectively reused for clothing classification tasks.

Optimization Details

- **Optimizer:** Adam
- **Loss Function:** BCEWithLogitsLoss
- **Evaluation Metric:** F1-score

Convergence Behavior

- Scratch models showed gradual improvement over multiple epochs
- Transfer learning models achieved high performance within a few epochs

Key Insight

Transfer learning significantly accelerates convergence by starting from a well-initialized feature space, reducing both training time and computational cost.

Overfitting Behavior

Overfitting was observed in later epochs, particularly in transfer learning models.

Indicators:

- Increasing training F1-score
- Plateau or slight decline in validation F1-score

Mitigation Strategies:

- Early stopping
- Data augmentation

Key Insight

Early stopping is essential in transfer learning setups, as models tend to converge quickly and may overfit if trained for too long.

4.7 Experimental Details

This section outlines the experimental setup used to train and evaluate all classification models.

Implementation Details

- **Framework:** PyTorch
- **Hardware:** Kaggle GPU (NVIDIA Tesla GPU)
- **Task:** Multi-label classification

Training Configuration

The following configurations were used across different models:

- **Loss Function:** BCEWithLogitsLoss
- **Optimizer:** Adam
- **Evaluation Metric:** F1-score (primary)

Key Insight

Binary Cross Entropy with logits is well-suited for multi-label classification as it independently evaluates each class without enforcing mutual exclusivity.

Training Setup per Model

- **ResNet-50 (Scratch):** 15 epochs
- **ResNet-50 (Transfer):** 5 epochs
- **EfficientNet (Scratch):** 15 epochs
- **EfficientNet (Transfer):** 7 epochs
- **MobileNetV3 (Scratch):** 20 epochs
- **MobileNetV3 (Transfer):** 7 epochs

Key Insight

Transfer learning models require significantly fewer epochs to converge compared to models trained from scratch.

Training Time Analysis

A clear difference in training time was observed:

- **ResNet-50:** $\sim$28 minutes per epoch
- **MobileNetV3:** $\sim$6 minutes per epoch

Key Insight

Lightweight architectures significantly reduce training time, making them ideal for rapid experimentation and deployment in resource-constrained environments.

Threshold Tuning

Multiple thresholds were evaluated for converting predicted probabilities into binary labels:

- 0.3 $\rightarrow$ Macro F1: 0.8227

- 0.4 → Macro F1: **0.8246** (Best)
- 0.5 → Macro F1: 0.8232
- 0.6 → Macro F1: 0.8192

Key Insight

A threshold of 0.4 provides the best balance between precision and recall, highlighting the importance of threshold tuning in multi-label classification.

4.8 Results

Quantitative Results

Model	Training Type	Epochs	Best Val F1
ResNet-50	Scratch	15	0.7833
ResNet-50	Transfer	5	**0.8666**
EfficientNet	Scratch	15	0.7141
EfficientNet	Transfer	7	0.8481
MobileNetV3	Scratch	20	0.6685
MobileNetV3	Transfer	7	0.8233

Table 4: Comparison of classification performance across different models and training strategies

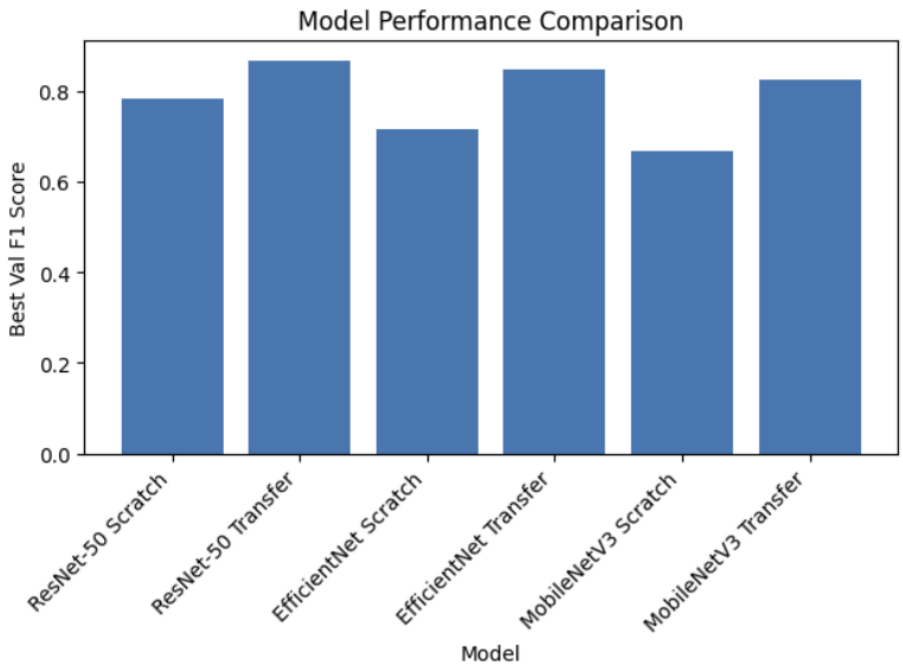

Figure 1: Comparison of final F1-scores across different models

Key Insight

ResNet-50 with transfer learning achieves the highest performance, demonstrating the effectiveness of deep architectures combined with pretrained feature representations.

Per-Class Results: Best Model (ResNet-50 Transfer Learning)

Category	Precision	Recall	F1-Score
Short Sleeve Top	0.9088	0.8825	0.8955
Trousers	**0.9171**	**0.9179**	**0.9175**
Shorts	0.8410	0.8316	0.8363
Long Sleeve Top	0.8332	0.8071	0.8200
Skirt	0.9033	0.8392	0.8701
Macro Average	0.8807	0.8557	0.8679
Micro Average	0.8910	0.8669	0.8788

Table 5: Per-class classification performance for ResNet-50 (Transfer Learning)

Observation:

- Trousers achieve the highest performance across all metrics (F1 = 0.9175)
- Short Sleeve Top and Skirt also show strong performance
- Long Sleeve Top is relatively more challenging due to visual similarity with other categories

Analysis:

- High precision and recall indicate strong feature representation learned by the model
- Performance variation across classes highlights the effect of class imbalance and visual overlap

Key Insight

ResNet-50 with transfer learning demonstrates robust and consistent performance across all clothing categories, making it the most effective classification model in this study.

Detailed Metrics (MobileNetV3 Transfer)

The MobileNetV3 model with transfer learning achieved strong performance while maintaining computational efficiency.

- **Precision:** 0.8403
- **Recall:** 0.8072
- **F1-score:** 0.8233
- **ROC-AUC:** 0.9501

Key Insight

MobileNetV3 achieves competitive performance with significantly lower computational cost, making it suitable for real-time and resource-constrained applications.

EfficientNet (Transfer Learning) Performance

EfficientNet with transfer learning demonstrated strong performance with rapid convergence.

- Best Validation F1-score: **0.8481**
- Converged within 3–4 epochs
- Stable performance across training

Key Insight

EfficientNet benefits significantly from transfer learning, achieving high performance while maintaining better parameter efficiency compared to deeper architectures.

4.9 Comparative Analysis

A comprehensive comparison was conducted across all models and training strategies to evaluate performance, efficiency, and convergence behavior.

Effect of Transfer Learning

Transfer learning consistently outperformed training from scratch across all architectures.

- **ResNet-50:** +8% improvement in F1-score
- **MobileNetV3:** +15% improvement in F1-score

Key Insight

Pretrained models leverage rich feature representations learned from large-scale datasets, resulting in faster convergence and significantly improved performance.

Effectiveness of EfficientNet Transfer Learning

EfficientNet demonstrated strong performance when initialized with pretrained weights.

- Rapid convergence within early epochs
- Stable validation performance
- High F1-score with fewer parameters

Key Insight

EfficientNet highlights the importance of architecture efficiency combined with transfer learning, achieving strong performance with reduced computational cost.

Architecture Comparison

- **ResNet-50:** Achieved the highest performance due to its deep architecture and strong feature extraction capability
- **MobileNetV3:** Provided competitive performance with significantly lower computational cost

- **EfficientNet:** Underperformed in the scratch setting due to lack of pretrained initialization

Key Insight

Deeper architectures capture more complex hierarchical features, while lightweight models prioritize efficiency, leading to a trade-off between accuracy and computational cost.

Training Efficiency

- ResNet required significantly more training time
- MobileNet enabled faster experimentation and iteration

Convergence Behavior

- Scratch models: Slow and gradual improvement across epochs
- Transfer learning models: Rapid convergence within the initial epochs

Key Insight

Transfer learning reduces the need for extensive training by starting from a well-initialized feature space, enabling faster and more stable convergence.

4.10 Discussion and Insights

This section presents key observations and insights derived from the experiments.

Key Insights

- Transfer learning is essential for achieving high performance in multi-label classification tasks
- Deep architectures such as ResNet capture complex visual patterns more effectively
- Lightweight models like MobileNetV3 provide a strong balance between efficiency and accuracy
- EfficientNet with transfer learning achieves near state-of-the-art performance with better efficiency

Key Insight

The combination of deep architectures and pretrained weights yields the best performance, while lightweight models offer practical deployment advantages.

Overfitting Behavior

Overfitting was observed in transfer learning models after a few epochs.

Evidence:

- Training F1-score continues to increase
- Validation F1-score plateaus or slightly decreases

Key Insight

Early stopping plays a crucial role in preventing overfitting, especially in transfer learning scenarios where models converge rapidly.

Failure Cases

Several failure scenarios were identified:

- **Occlusion:** Clothing items partially hidden by other objects
- **Small Objects:** Difficulty detecting small-scale garments
- **Class Confusion:** Visually similar categories (e.g., tops vs dresses)

Key Insight

Most failure cases arise due to challenging visual conditions rather than fundamental limitations of the model architecture.

Trade-offs

- Accuracy vs Computational Efficiency
- Training Time vs Performance

Effectiveness of Transfer Learning

Transfer learning significantly improves performance by leveraging pretrained representations learned from large-scale datasets.

- Early layers capture generic features such as edges and textures
- Later layers adapt to task-specific patterns such as clothing structures

Key Insight

The reuse of pretrained features reduces the need for large datasets and enables faster convergence.

Qualitative Analysis

Qualitative analysis revealed that:

- Models perform well on clear and well-lit images
- Performance degrades in complex scenes with multiple overlapping objects

Key Insight

Model performance is highly dependent on image quality and scene complexity, highlighting the importance of diverse training data.

4.11 Conclusion

The classification experiments demonstrate that transfer learning significantly enhances performance and reduces training time. Among all models, ResNet-50 with transfer learning achieved the best results, while MobileNetV3 provided an efficient alternative with competitive performance.

5 Detection and Segmentation

5.1 Problem Definition

The task is formulated as a **detection and instance segmentation problem**, where the objective is to identify all clothing items present in an image, classify them into predefined categories, and determine their precise spatial locations along with pixel-level segmentation.

Unlike classification tasks, which assign a single label to an entire image, this problem requires the model to simultaneously perform:

- **Object Localization** (detect where objects are present)
- **Object Classification** (identify the category of each object)
- **Pixel-wise Segmentation** (predict precise object boundaries)

Formally, given an input image $I \in R^{H \times W \times 3}$, the goal is to predict a set of object instances:

$$\mathcal{O} = \{(b_i, c_i, m_i)\}_{i=1}^{N}$$

where:

- $b_i = (x_{min}, y_{min}, x_{max}, y_{max})$ represents the bounding box coordinates
- $c_i \in \{1, \ldots, C\}$ represents the class label
- $m_i \in \{0, 1\}^{H \times W}$ represents the segmentation mask
- N is the number of detected instances

Key Insight

Instance segmentation is a superset of object detection and semantic segmentation, as it requires identifying individual objects while also predicting their exact pixel-level boundaries.

Input and Output Specification

Input: An image containing one or more persons wearing multiple clothing items under varying real-world conditions.

Output:

- Bounding boxes indicating the location of each clothing item
- Category labels corresponding to clothing types
- Segmentation masks representing precise object boundaries

Task Complexity

This task is significantly more complex than classification due to the following reasons:

- **Multiple Objects:** Each image may contain several clothing items
- **Instance Separation:** Different objects of the same class must be distinguished
- **Spatial Precision:** Accurate mask prediction requires pixel-level understanding

Key Insight

The requirement to detect, classify, and segment multiple instances simultaneously makes this task computationally intensive and algorithmically challenging.

Challenges in Detection and Segmentation

- **Occlusion:** Clothing items may be partially hidden by other objects
- **Scale Variation:** Objects appear at different sizes within the image
- **Complex Backgrounds:** Real-world scenes introduce noise and distractions
- **Fine Boundaries:** Accurate segmentation requires precise edge detection
- **Class Similarity:** Certain clothing categories share similar visual features

Key Insight

Among these challenges, occlusion and fine boundary detection are the most critical factors affecting segmentation performance.

Real-World Relevance

This formulation is highly relevant for practical applications such as:

- Virtual try-on systems
- Fashion recommendation engines
- Retail analytics
- Automated wardrobe analysis

Key Insight

Accurate instance segmentation enables fine-grained understanding of clothing items, which is essential for real-world fashion applications.

5.2 Dataset Preparation

The DeepFashion2 dataset was used for the detection and segmentation task. It provides rich annotations including bounding boxes, category labels, and polygon-based segmentation masks for each clothing instance.

Unlike classification, where labels are image-level, this task requires **instance-level annotations**, making data preparation significantly more complex.

Key Insight

Instance-level annotations enable the model to distinguish between multiple objects within the same image, which is essential for detection and segmentation tasks.

Annotation Format

Each image in the dataset contains structured annotations for multiple clothing instances. For each object, the following information is provided:

- **Bounding Box Coordinates:** $(x_{min}, y_{min}, x_{max}, y_{max})$
- **Category Label:** Clothing type (e.g., top, trousers)
- **Polygon Segmentation:** Set of coordinates defining object boundaries

Key Insight

Polygon annotations provide fine-grained boundary information, which is crucial for generating accurate segmentation masks.

Bounding Box Extraction

Bounding boxes were directly extracted from annotation files and used as ground truth for detection models.

$$b = (x_{min}, y_{min}, x_{max}, y_{max})$$

These coordinates define the rectangular region enclosing each clothing item.

Importance:

- Enables object localization
- Forms the basis for detection models like YOLO and Mask R-CNN

Segmentation Mask Generation

Polygon annotations were converted into pixel-wise binary masks.

$$M(x, y) = \begin{cases} 1 & \text{if pixel belongs to object} \\ 0 & \text{otherwise} \end{cases}$$

For each instance:

- Polygon coordinates were rasterized into masks
- Each mask corresponds to a single clothing item

Key Insight

Accurate mask generation is critical, as errors at this stage directly propagate to segmentation performance.

Instance-Level Representation

Each image may contain multiple clothing items, requiring instance-wise representation:

$$\mathcal{O} = \{(b_i, c_i, m_i)\}_{i=1}^{N}$$

This ensures:

- Multiple objects are handled independently
- Overlapping instances are correctly represented

Key Insight

Instance-level representation is essential for distinguishing between multiple objects of the same class.

Dataset Challenges

The dataset presents several real-world challenges:

- **Occlusion:** Objects overlap significantly
- **Scale Variation:** Clothing items appear at different sizes
- **Complex Backgrounds:** Non-uniform backgrounds introduce noise
- **Class Imbalance:** Some categories dominate others

Key Insight

These challenges make detection and segmentation tasks significantly harder than classification, requiring robust models and careful preprocessing.

Data Filtering and Selection

To ensure stable training:

- Images with extremely small or unclear objects were filtered
- Focus was placed on well-represented categories
- Extremely rare classes were excluded

Key Insight

Filtering noisy and rare samples improves training stability and helps the model generalize better.

5.3 Preprocessing Pipeline

Preprocessing plays a crucial role in preparing the dataset for detection and segmentation tasks. Unlike classification, preprocessing here must preserve both spatial structure and annotation alignment.

Key Insight

In detection and segmentation tasks, preprocessing must ensure that both images and their corresponding annotations (bounding boxes and masks) remain spatially consistent.

Image Resizing

All images were resized to a fixed resolution to ensure compatibility with deep learning models.

$$I' = Resize(I, H', W')$$

Key Consideration:

Unlike classification, resizing affects:

- Bounding box coordinates
- Segmentation masks

Thus, corresponding transformations were applied:

$$b' = \left(\frac{x_{min}}{W} \cdot W', \frac{y_{min}}{H} \cdot H', \frac{x_{max}}{W} \cdot W', \frac{y_{max}}{H} \cdot H'\right)$$

Key Insight

Incorrect scaling of bounding boxes leads to misaligned training labels, severely degrading detection performance.

Normalization

Pixel values were normalized using ImageNet statistics:

$$I_{norm} = \frac{I - \mu}{\sigma}$$

where μ and σ represent mean and standard deviation.

Purpose:

- Stabilizes training
- Improves convergence
- Matches pretrained model expectations

Data Augmentation

To improve generalization, several augmentation techniques were applied:

- **Horizontal Flipping**
- **Rotation**
- **Brightness Adjustment**

Important Constraint:

All augmentations were applied consistently to:

- Images
- Bounding boxes
- Segmentation masks

Key Insight

Inconsistent augmentation between images and annotations leads to incorrect supervision and degraded model performance.

Bounding Box Transformation under Augmentation

Example: Horizontal Flip

$$x' = W - x$$

Bounding boxes were updated accordingly to maintain spatial correctness.

Mask Transformation

Segmentation masks were transformed using the same augmentation pipeline as images.

- Rotation applied to masks
- Flipping applied pixel-wise

Key Insight

Mask transformations must preserve object boundaries to maintain segmentation accuracy.

Pipeline Summary

The complete preprocessing pipeline is summarized as follows:

1. Load image and annotations
2. Resize image
3. Scale bounding boxes
4. Generate masks from polygons
5. Apply normalization
6. Apply augmentations (image + annotations)

Key Insight

A robust preprocessing pipeline ensures that the model learns meaningful spatial and structural patterns from the data.

5.4 Model Architectures and Training Strategies

To address the detection and segmentation task, three different models were explored, each representing a different level of visual understanding:

- U-Net (Semantic Segmentation)
- YOLO (Object Detection + Segmentation)
- Mask R-CNN (Instance Segmentation)

These models differ significantly in their design philosophy and capabilities:

- **U-Net:** Focuses on pixel-level segmentation without distinguishing individual instances
- **YOLO:** Performs fast object detection with approximate segmentation
- **Mask R-CNN:** Provides precise instance-level segmentation along with detection

Key Insight

Using multiple models allows a comprehensive evaluation of different levels of visual understanding, from pixel-level segmentation to instance-level detection and segmentation.

Each model was trained using two strategies:

- Training from scratch
- Transfer learning using pretrained weights

This enables a detailed comparison of how pretrained knowledge impacts performance across different architectures.

5.4.1 U-Net (Semantic Segmentation)

U-Net is a convolutional neural network designed for **pixel-wise semantic segmentation**. It assigns a class label to each pixel in the image, enabling fine-grained understanding of object regions.

In this project, U-Net is used to segment clothing regions from images, identifying which pixels belong to clothing items and which belong to the background.

Key Insight

Unlike detection models, U-Net focuses purely on pixel-level classification and does not distinguish between multiple instances of the same object.

Problem Formulation The segmentation task can be formulated as a mapping from an input image to a pixel-wise mask.

Given an input image:

$$I \in R^{H \times W \times 3}$$

the model predicts a segmentation mask:

$$M \in \{0,1\}^{H \times W}$$

where each pixel is classified as:

$$M(x,y) = \begin{cases} 1 & \text{if pixel belongs to a clothing region} \\ 0 & \text{otherwise} \end{cases}$$

Key Insight

This formulation converts the problem from image-level classification to pixel-level prediction, significantly increasing complexity.

Nature of the Task Unlike classification or detection:

- Every pixel contributes to the prediction
- Spatial relationships between pixels are crucial
- Boundary accuracy becomes important

Key Insight

Segmentation requires both high-level semantic understanding and low-level spatial precision.

Architecture Overview U-Net follows an **encoder-decoder architecture**, consisting of two main components:

- **Encoder (Contracting Path)**
- **Decoder (Expanding Path)**

The encoder extracts high-level feature representations, while the decoder reconstructs spatial information to produce a pixel-wise segmentation mask.

Key Insight

The encoder captures semantic information, while the decoder restores spatial details required for precise segmentation.

Encoder (Contracting Path) The encoder is responsible for extracting hierarchical features from the input image.

Operations:

- Convolution layers
- Activation (ReLU)
- Max pooling (downsampling)

At each stage, the spatial resolution decreases while the number of feature channels increases.

$$X_{l+1} = \text{MaxPool}(\sigma(W_l * X_l + b_l))$$

where:

- $*$ denotes convolution
- σ is the activation function

Effect:

- Captures high-level semantic features
- Reduces spatial dimensions

Key Insight

The encoder transforms the image into a compact feature representation capturing object-level information.

Decoder (Expanding Path) The decoder reconstructs the segmentation mask by progressively increasing spatial resolution.

Operations:

- Upsampling (transposed convolution)
- Convolution layers

$$X_{l-1} = \text{UpConv}(X_l)$$

Effect:

- Restores spatial resolution
- Produces dense pixel predictions

Key Insight

The decoder converts compressed features back into a full-resolution segmentation map.

Skip Connections A key feature of U-Net is the use of skip connections, which directly connect encoder layers to corresponding decoder layers.

$$X_{dec} = \text{Concat}(X_{enc}, X_{up})$$

Purpose:

- Preserve fine-grained spatial details
- Improve boundary accuracy

Key Insight

Skip connections help retain low-level features such as edges, which are essential for accurate segmentation boundaries.

Why U-Net for This Task U-Net is particularly suitable for clothing segmentation because:

- It captures both global context and local details
- It produces precise pixel-level predictions
- It performs well even with limited data

Key Insight

U-Net's architecture is well-suited for segmentation tasks where boundary precision and spatial detail are critical.

Dataset Visualization and Mask Generation To better understand the segmentation task, visualizations of the dataset were generated, including the original image, corresponding segmentation mask, and overlay.

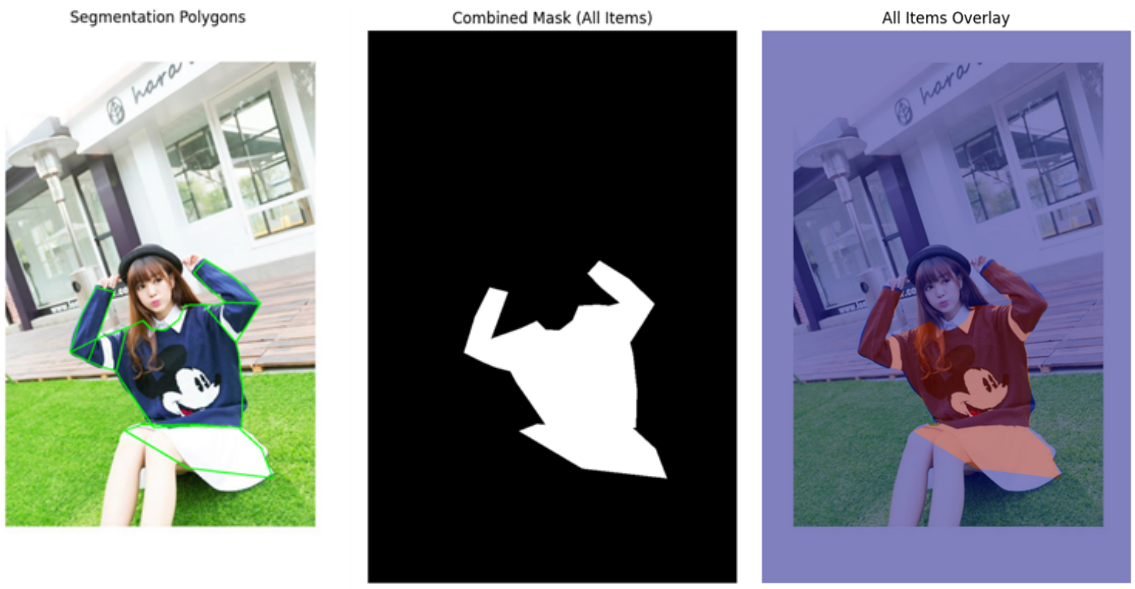

Figure 2: Original image, ground truth segmentation mask, and overlay visualization

Observation: The segmentation mask accurately highlights clothing regions present in the image.

Analysis: The overlay demonstrates that the generated masks are well-aligned with the clothing items, confirming correct preprocessing and annotation handling.

Insight: High-quality masks are essential for training segmentation models, as inaccuracies in masks directly impact model performance.

Mask Representation Segmentation masks are represented as binary matrices:

$$M(x, y) = \begin{cases} 1 & \text{if pixel belongs to clothing} \\ 0 & \text{otherwise} \end{cases}$$

Each pixel contributes to the loss function, making segmentation a dense prediction problem.

Key Insight

Unlike classification, segmentation requires prediction for every pixel, significantly increasing computational complexity.

Multi-Class Mask Representation In addition to binary masks, multi-class segmentation masks were also considered, where each pixel is assigned a class label corresponding to a specific clothing category.

$$M(x, y) \in \{0, 1, 2, \ldots, C\}$$

where:

- 0 represents background
- $1 \ldots C$ represent different clothing categories

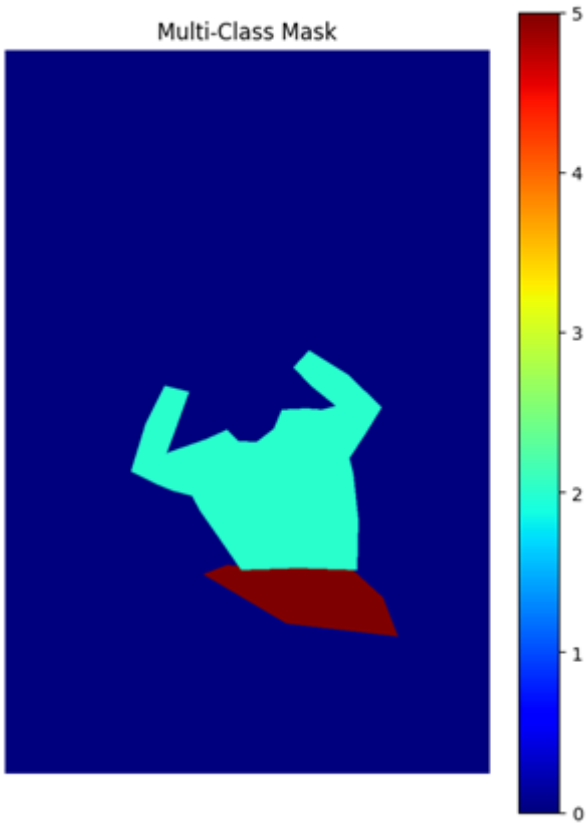

Figure 3: Multi-class segmentation mask with different colors representing different clothing categories

Observation: Different clothing categories are represented using distinct colors in the mask.

Analysis: This representation provides more detailed information compared to binary masks, enabling category-wise segmentation.

Insight: Multi-class segmentation increases task complexity, as the model must distinguish between multiple categories at the pixel level.

> Key Insight
>
> Multi-class segmentation provides richer information but requires more complex learning compared to binary segmentation.

Choice of Mask Representation Although multi-class masks provide richer information, the primary experiments in this project were conducted using binary masks for U-Net.

Reason:

- Simpler learning objective
- Faster convergence
- Reduced computational complexity

> Key Insight
>
> Binary segmentation simplifies the problem, allowing the model to focus on accurate region detection rather than category differentiation.

Challenges Observed From dataset visualization, several challenges were identified:

- **Occlusion:** Clothing items overlap
- **Complex Backgrounds:** Background noise affects mask clarity
- **Fine Boundaries:** Edges of clothing are difficult to capture
- **Small Objects:** Smaller garments are harder to segment

> Key Insight
>
> Among these challenges, boundary precision and occlusion are the most significant factors affecting segmentation accuracy.

Training Methodology To evaluate the performance of U-Net, two training strategies were employed:

- Training from scratch
- Transfer learning using pretrained encoders

This comparison allows us to analyze the impact of pretrained feature representations on segmentation performance.

> Key Insight
>
> Transfer learning plays a critical role in segmentation tasks by providing strong initial feature representations.

Training from Scratch In this approach, all parameters of the U-Net model were initialized randomly and trained end-to-end using the dataset.

Characteristics:

- No prior knowledge
- Learns features directly from dataset
- Requires more epochs for convergence

Observed Behavior:

- Slow convergence during training
- Higher training loss in early epochs
- Difficulty capturing fine-grained spatial details

> Key Insight
>
> Without pretrained features, the model must learn both low-level and high-level representations from scratch, which is computationally expensive and less effective.

Transfer Learning In transfer learning, the encoder part of U-Net was initialized with pretrained weights (e.g., ImageNet-trained backbone).

Approach:

- Initialize encoder with pretrained weights
- Train decoder layers
- Fine-tune entire network

Advantages:

- Faster convergence
- Better feature extraction
- Improved segmentation accuracy

Observed Behavior:

- Rapid improvement in early epochs
- Lower validation loss
- Better boundary detection

> Key Insight
>
> Pretrained encoders already capture essential visual features such as edges and textures, significantly improving segmentation performance.

Loss Function The model was trained using a combination of Binary Cross Entropy (BCE) and Dice Loss:

$$\mathcal{L} = \mathcal{L}_{BCE} + \mathcal{L}_{Dice}$$

where:

$$\mathcal{L}_{Dice} = 1 - \frac{2|P \cap G|}{|P| + |G|}$$

Why this combination?

- BCE ensures pixel-wise classification accuracy
- Dice loss improves overlap between prediction and ground truth

Key Insight

Combining BCE and Dice loss balances pixel accuracy with region overlap, making it highly effective for segmentation tasks.

Convergence Behavior A clear difference in convergence was observed:

- Scratch model: Gradual improvement over epochs
- Transfer model: Rapid convergence within fewer epochs

Key Insight

Transfer learning reduces training time while achieving significantly better performance.

Overfitting Behavior In later epochs, slight overfitting was observed:

- Training performance continued to improve
- Validation performance plateaued

Mitigation Strategies:

- Data augmentation
- Early stopping

Key Insight

Monitoring validation performance is essential to prevent overfitting in segmentation models.

Results To evaluate the performance of U-Net, both scratch and transfer learning approaches were compared using standard segmentation metrics.

Model	**IoU**	**Dice Score**
U-Net (Scratch)	0.3137	0.3000
U-Net (Transfer)	**0.4870**	**0.5575**

Table 6: Performance comparison of U-Net trained from scratch and with transfer learning

Quantitative Results **Observation:** Transfer learning improves both IoU and Dice score compared to training from scratch.

Performance Improvement Analysis

- IoU improvement: ∼55% increase
- Dice improvement: ∼85% increase

Interpretation:

- The scratch model struggles to learn meaningful spatial features
- Transfer learning significantly enhances feature representation
- Despite improvement, absolute performance remains moderate

> Key Insight
>
> Transfer learning provides substantial improvement, but U-Net still faces limitations in complex scenes, particularly due to lack of instance-level understanding.

Discussion and Insights The experimental results demonstrate that U-Net is capable of learning meaningful spatial representations for segmentation tasks. However, its performance is influenced by both architectural limitations and dataset complexity.

Key Observations

- Transfer learning significantly improves segmentation performance
- The model is able to capture general clothing regions effectively
- Boundary precision improves with pretrained encoders

> Key Insight
>
> Pretrained feature extractors play a crucial role in improving segmentation performance, especially when training data is limited.

Limitations of U-Net Despite its effectiveness, several limitations were observed:

- **Lack of Instance Awareness:** U-Net performs semantic segmentation and cannot distinguish between multiple instances of the same class.
- **Merged Predictions:** Multiple clothing items are often combined into a single region.
- **Difficulty with Overlapping Objects:** Overlapping garments are not properly separated.
- **Moderate Performance:** Even with transfer learning, segmentation accuracy remains limited compared to more advanced models.

> Key Insight
>
> The inability to perform instance-level segmentation is the most critical limitation of U-Net in complex real-world scenarios.

Failure Analysis From qualitative results, several failure cases were observed:

- Small clothing items are often missed
- Boundaries between adjacent objects are not clearly defined

- Occluded regions lead to incomplete segmentation

Key Insight

These failure cases highlight the need for models that can perform both detection and segmentation simultaneously.

Overall Interpretation

- U-Net is effective for general region segmentation
- Transfer learning improves performance significantly
- However, the model lacks the ability to distinguish individual object instances

Key Insight

While U-Net provides a strong baseline for segmentation, it is insufficient for tasks requiring instance-level understanding, motivating the use of advanced models such as Mask R-CNN.

5.4.2 YOLO (Detection and Segmentation)

YOLO (You Only Look Once) is a **single-stage object detection model** that performs object localization and classification in a single forward pass. Unlike traditional two-stage detectors, YOLO directly predicts bounding boxes and class probabilities from the entire image.

In this project, YOLO is used for:

- Detecting clothing items (bounding boxes)
- Classifying clothing categories
- Generating approximate segmentation masks

Key Insight

YOLO treats object detection as a regression problem, enabling fast and efficient predictions suitable for real-time applications.

Problem Formulation Given an input image:

$$I \in R^{H \times W \times 3}$$

YOLO predicts a set of detections:

$$\mathcal{D} = \{(b_i, c_i, s_i)\}_{i=1}^{N}$$

where:

- $b_i = (x, y, w, h)$ represents bounding box coordinates
- c_i represents class probabilities
- s_i represents object confidence score

Final prediction score:

$$Score = P(\text{object}) \times P(\text{class})$$

Key Insight

YOLO predicts both localization and classification simultaneously, making it significantly faster than traditional detection methods.

Grid-Based Detection Mechanism YOLO divides the input image into an $S \times S$ grid.

Each grid cell predicts:

- Bounding boxes
- Confidence scores
- Class probabilities

Key Insight

Each grid cell is responsible for detecting objects whose center falls within that cell.

Nature of the Task Compared to U-Net:

- YOLO performs object-level prediction instead of pixel-level
- It identifies **where** objects are present
- It does not provide highly detailed segmentation boundaries

Key Insight

YOLO focuses on speed and detection accuracy, but sacrifices fine-grained segmentation precision.

Architecture Overview YOLO follows a **single-stage detection architecture**, where the entire image is processed in a single forward pass to predict bounding boxes, class probabilities, and object confidence scores.

The architecture consists of three main components:

- **Backbone Network**
- **Neck (Feature Aggregation)**
- **Detection Head**

Key Insight

Unlike two-stage detectors, YOLO performs detection in a single pass, making it significantly faster and suitable for real-time applications.

Backbone Network The backbone is responsible for extracting feature representations from the input image.

Function:

- Extract low-level features (edges, textures)
- Extract high-level semantic features

$$F = \phi(I)$$

where:

- I = input image
- ϕ = feature extraction network
- F = feature map

Key Insight

The quality of feature extraction directly impacts detection accuracy.

Neck (Feature Aggregation) The neck combines features from different layers to capture multi-scale information.

Purpose:

- Detect objects of varying sizes
- Combine spatial and semantic features

Key Insight

Multi-scale feature aggregation helps YOLO detect both small and large objects effectively.

Detection Head The detection head predicts:

- Bounding box coordinates
- Object confidence scores
- Class probabilities

Each prediction corresponds to a potential object instance.

$$\hat{y} = (b, c, s)$$

Bounding Box Representation Each bounding box is represented as:

$$b = (x, y, w, h)$$

where:

- (x, y) = center coordinates
- (w, h) = width and height

Key Insight

Accurate bounding box prediction is critical for correct object localization.

Loss Function YOLO optimizes a multi-component loss function:

$$L = \lambda_{coord} L_{box} + L_{obj} + L_{cls}$$

where:

- L_{box} = bounding box regression loss
- L_{obj} = objectness loss
- L_{cls} = classification loss

Interpretation:

- L_{box} ensures accurate localization
- L_{obj} ensures correct object presence prediction
- L_{cls} ensures correct classification

Key Insight

The combined loss enables YOLO to jointly optimize detection and classification in a single framework.

Segmentation Extension (YOLOv8) YOLOv8 introduces a segmentation head that predicts masks along with bounding boxes.

Output:

- Bounding boxes
- Class labels
- Segmentation masks

Key Insight

While YOLO supports segmentation, its masks are less precise compared to dedicated segmentation models.

Model Outputs and Visualization To evaluate the performance of YOLO, both scratch and transfer learning models were analyzed using prediction outputs and ROC curves.

Prediction Results **Observation:** The transfer learning model produces more accurate detections with better bounding box alignment.

Analysis: Scratch model shows missed detections and less precise localization, whereas transfer learning improves both detection confidence and accuracy.

Insight: Pretrained weights significantly enhance detection capability, especially in complex scenes.

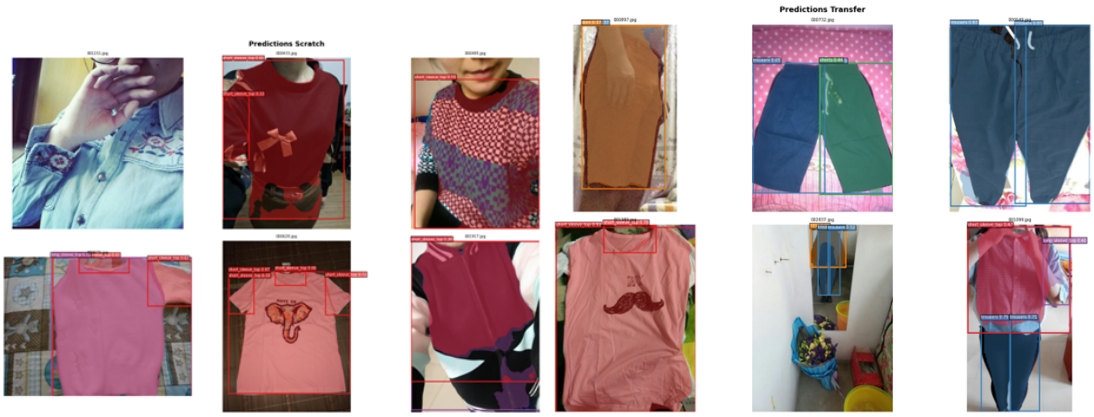

Figure 4: YOLO predictions: Scratch (left) vs Transfer Learning (right)

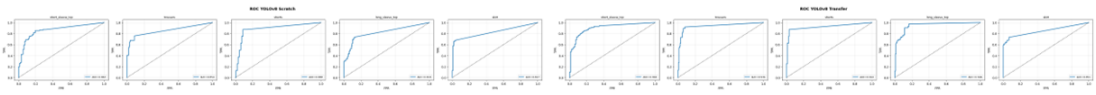

Figure 5: ROC curves for YOLO: Scratch vs Transfer Learning

ROC Curve Analysis **Observation:** The ROC curve for the transfer learning model dominates the scratch model.

Analysis: Higher true positive rates at lower false positive rates indicate improved classification performance.

Insight: Transfer learning enhances the model's ability to distinguish between object and background.

Detection Quality Analysis

- Transfer learning improves bounding box precision
- Scratch model struggles with small or occluded objects
- Detection confidence is higher in transfer learning

Key Insight

Visual and ROC analysis together confirm that transfer learning significantly improves YOLO performance.

Training Methodology YOLO models were trained using two strategies:

- Training from scratch
- Transfer learning using pretrained weights

Key Insight

Transfer learning is particularly effective in detection tasks, as pretrained models already capture rich visual features.

Training from Scratch In this approach, the YOLO model was initialized with random weights and trained on the dataset.

Characteristics:

- No prior knowledge
- Requires more training iterations
- Slower convergence

Observed Behavior:

- Lower detection accuracy
- Missed detections, especially for small objects
- Less stable training

> **Key Insight**
>
> Without pretrained features, the model struggles to learn robust object representations.

Transfer Learning In transfer learning, the YOLO model was initialized with pretrained weights and fine-tuned on the dataset.

Advantages:

- Faster convergence
- Improved detection accuracy
- Better generalization

Observed Behavior:

- Higher confidence predictions
- Improved localization
- Better detection of small and occluded objects

> **Key Insight**
>
> Pretrained models provide strong feature representations, significantly improving detection performance.

Evaluation Metrics The performance of YOLO was evaluated using:

- **mAP@0.5:** Mean Average Precision at IoU threshold 0.5
- **mAP@[0.5:0.95]:** Average precision across multiple IoU thresholds

> **Key Insight**
>
> mAP@[0.5:0.95] is a stricter metric, providing a more comprehensive evaluation of detection performance.

Quantitative Results **Observation:** Transfer learning improves both mAP metrics significantly.

Model	mAP@0.5	mAP@[0.5:0.95]
YOLOv8m (Scratch)	0.58	0.42
YOLOv8m (Transfer)	**0.69**	**0.52**

Table 7: YOLO detection performance comparison

Performance Analysis

- Transfer learning improves detection accuracy across all IoU thresholds
- Scratch model struggles with precise localization
- Improvement is more pronounced in stricter metric (mAP@[0.5:0.95])

Interpretation:

- Pretrained features enhance both classification and localization
- Transfer learning helps model generalize better to unseen data

Key Insight

The improvement in mAP@[0.5:0.95] indicates better bounding box precision and localization accuracy.

ROC Curve Interpretation The ROC curves further validate the performance improvement:

- Transfer learning achieves higher true positive rates
- Lower false positive rates compared to scratch model

Key Insight

ROC analysis confirms that transfer learning enhances classification confidence in YOLO.

Discussion and Insights The experimental results demonstrate that YOLO is highly effective for object detection tasks, particularly in terms of speed and efficiency. However, its performance varies depending on the complexity of the scene and the quality of feature representations.

Key Observations

- Transfer learning significantly improves detection performance
- The model efficiently detects multiple objects in a single image
- Localization accuracy improves with pretrained weights

Key Insight

YOLO achieves strong detection performance while maintaining high computational efficiency, making it suitable for real-time applications.

Strengths of YOLO

- **Fast Inference:** Single-stage architecture enables real-time detection.
- **End-to-End Learning:** Simultaneously predicts bounding boxes and class labels.

- **Efficient for Multiple Objects:** Can detect multiple clothing items in a single forward pass.

> **Key Insight**
>
> YOLO is optimized for speed and efficiency, making it highly suitable for applications requiring real-time detection.

Limitations of YOLO Despite its strengths, several limitations were observed:

- **Limited Segmentation Precision:** The segmentation outputs are coarse and lack fine boundary details.
- **Difficulty with Small Objects:** Smaller clothing items are sometimes missed.
- **Overlapping Objects:** Struggles to accurately separate overlapping garments.
- **Localization Errors:** Bounding boxes may not tightly fit objects in complex scenes.

> **Key Insight**
>
> The primary limitation of YOLO lies in its inability to produce precise segmentation masks, as it prioritizes speed over fine-grained accuracy.

Failure Analysis From qualitative results, the following failure cases were identified:

- Missed detections in cluttered backgrounds
- Incorrect classification for visually similar categories
- Partial detection of occluded objects

> **Key Insight**
>
> These failure cases highlight the limitations of single-stage detectors in handling complex visual scenarios.

Overall Interpretation

- YOLO provides strong detection performance with high efficiency
- Transfer learning significantly enhances model capability
- However, segmentation quality and instance separation remain limited

> **Key Insight**
>
> While YOLO is highly effective for detection tasks, it lacks the precision required for detailed segmentation, motivating the use of more advanced models such as Mask R-CNN.

5.4.3 Mask R-CNN (Instance Segmentation)

Mask R-CNN is a state-of-the-art deep learning model designed for **instance segmentation**, which combines object detection and pixel-wise segmentation.

Unlike U-Net and YOLO:

- U-Net performs semantic segmentation (no instance separation)

- YOLO performs object detection (coarse segmentation)
- Mask R-CNN performs **instance-level segmentation**

This makes Mask R-CNN the most comprehensive model for this task.

Key Insight

Mask R-CNN simultaneously detects objects, classifies them, and generates precise segmentation masks for each individual instance.

Problem Formulation Given an input image:

$$I \in R^{H \times W \times 3}$$

the model predicts a set of instances:

$$\mathcal{O} = \{(b_i, c_i, m_i)\}_{i=1}^{N}$$

where:

- b_i = bounding box of object i
- c_i = class label
- $m_i \in \{0, 1\}^{H \times W}$ = segmentation mask

Key Insight

Unlike semantic segmentation, instance segmentation assigns a unique mask to each object, even if they belong to the same class.

Why Instance Segmentation is Important In real-world clothing images:

- Multiple garments are present
- Objects overlap frequently
- Each clothing item must be individually identified

Example:

- Shirt + Jacket + Pants $\rightarrow$ 3 separate masks

Key Insight

Instance segmentation provides richer and more detailed information compared to both detection and semantic segmentation.

Nature of the Task Compared to previous models:

- More complex than classification and detection
- Requires both localization and pixel-level accuracy

- Handles multiple objects simultaneously

Key Insight

Instance segmentation is one of the most challenging tasks in computer vision due to the need for both precise localization and detailed segmentation.

Architecture Overview Mask R-CNN follows a **two-stage detection architecture**, extending Faster R-CNN by adding a segmentation branch.

The model consists of the following key components:

- Backbone Network (Feature Extraction)
- Region Proposal Network (RPN)
- ROI Align
- Detection Head (Classification + Bounding Box)
- Mask Head (Segmentation)

Key Insight

Mask R-CNN separates detection and segmentation into different branches, allowing more accurate and specialized predictions.

Backbone Network The backbone (e.g., ResNet + Feature Pyramid Network) extracts feature maps from the input image.

$$F = \phi(I)$$

where:

- I = input image
- ϕ = backbone network
- F = feature map

Key Insight

Feature Pyramid Networks enable multi-scale feature extraction, improving detection of both small and large objects.

Region Proposal Network (RPN) The RPN generates candidate object regions (proposals) from the feature map.

Output:

- Proposed bounding boxes
- Objectness scores

$$R = \text{RPN}(F)$$

Purpose:

- Identify regions likely to contain objects
- Reduce search space for detection

> **Key Insight**
>
> The RPN acts as a filter, selecting only relevant regions for further processing.

ROI Align ROI Align extracts fixed-size feature maps from each proposed region.

Key Difference from ROI Pooling:

- ROI Pooling introduces quantization errors
- ROI Align preserves exact spatial locations

$$f(x) = \sum w_i x_i$$

> **Key Insight**
>
> ROI Align is critical for achieving high-quality segmentation masks by preserving spatial precision.

Detection Head The detection head performs:

- Object classification
- Bounding box regression

Output:

- Class label (c)
- Bounding box (b)

Mask Head The mask head predicts a binary mask for each detected object.

$$m_i \in \{0, 1\}^{H \times W}$$

Characteristics:

- Pixel-wise prediction
- Separate mask for each instance

> **Key Insight**
>
> The mask head enables instance-level segmentation, distinguishing between multiple objects of the same class.

Loss Function Mask R-CNN optimizes a multi-task loss:

$$L = L_{cls} + L_{box} + L_{mask}$$

where:

- L_{cls} = classification loss
- L_{box} = bounding box regression loss
- L_{mask} = segmentation loss (binary cross-entropy)

Interpretation:

- L_{cls} ensures correct object classification
- L_{box} ensures accurate localization
- L_{mask} ensures precise segmentation

Key Insight

The multi-task loss enables Mask R-CNN to jointly optimize detection and segmentation, making it more powerful than single-task models.

Why Mask R-CNN for This Task Mask R-CNN is particularly suitable because:

- It detects multiple clothing items
- It generates precise segmentation masks
- It handles overlapping objects effectively

Key Insight

Mask R-CNN provides a complete solution by combining detection and segmentation, making it the most suitable model for this project.

Dataset Visualization and Model Outputs To evaluate the effectiveness of Mask R-CNN, visualizations of predictions were analyzed, including bounding boxes, class labels, and instance-level segmentation masks.

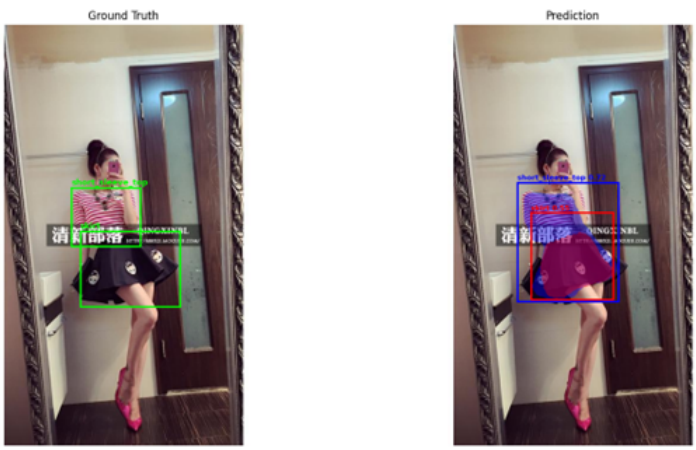

Figure 6: Mask R-CNN predictions showing bounding boxes and instance-level segmentation masks

Instance Segmentation Results **Observation:** The model successfully detects multiple clothing items and generates separate masks for each instance.

Analysis: Unlike U-Net, which merges all regions, Mask R-CNN distinguishes individual objects even when they belong to the same class.

Insight: Instance-level segmentation enables fine-grained understanding of complex scenes.

Key Insight

Mask R-CNN uniquely provides both object detection and precise segmentation for each individual instance.

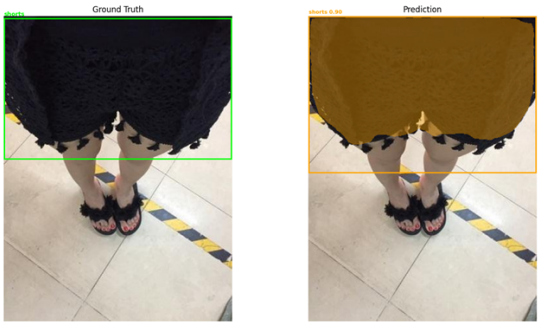

Figure 7: Ground truth vs Mask R-CNN prediction

Comparison with Ground Truth **Observation:** Predicted masks closely match ground truth masks in both shape and location.

Analysis: The model accurately captures object boundaries and separates overlapping instances.

Insight: Mask R-CNN achieves significantly higher segmentation precision compared to previous models.

Qualitative Improvements over Other Models

- Compared to U-Net: Separate masks instead of merged regions
- Compared to YOLO: More precise segmentation boundaries
- Better handling of overlapping objects

Key Insight

Mask R-CNN outperforms both U-Net and YOLO by combining accurate detection with high-quality segmentation.

Results and Evaluation

The performance of Mask R-CNN was evaluated using both detection and segmentation metrics to provide a comprehensive understanding of model effectiveness.

Detection Performance Detection results were evaluated at an IoU threshold of 0.5 using precision, recall, and F1-score.

Model	Precision	Recall	F1 Score
Mask R-CNN (Scratch)	0.5896	0.8771	0.7052
Mask R-CNN (Transfer)	**0.6149**	0.8740	**0.7219**

Table 8: Detection performance comparison (micro average)

Observation: Transfer learning provides a slight improvement in detection performance, particularly in precision and F1-score.

Segmentation Performance Segmentation quality was evaluated using mean Intersection over Union (mIoU) and Dice coefficient.

Model	mIoU	Dice Score
Mask R-CNN (Scratch)	0.8517	0.9168
Mask R-CNN (Transfer)	**0.8600**	**0.9218**

Table 9: Segmentation performance comparison

Observation: Both models achieve very high segmentation performance, with mIoU exceeding 0.85 across all classes.

Key Insight

Mask R-CNN demonstrates excellent segmentation capability, achieving highly accurate pixel-level predictions even without pretrained initialization.

Detection Quality (mAP)

Metric	Value
mAP@[0.5:0.95]	0.6409

Table 10: Mean Average Precision for Mask R-CNN (Scratch)

Observation: The model achieves strong detection performance across multiple IoU thresholds.

Class-wise Analysis

Class	AUC	Best F1
Short Sleeve Top	0.8991	0.8036
Trousers	0.9151	0.8136
Shorts	0.9043	0.6966
Long Sleeve Top	0.8314	0.6026
Skirt	0.8607	0.7052

Table 11: ROC-AUC and optimal F1-score per class

Observation:

Performance varies across classes, with trousers achieving the highest AUC, while long sleeve tops remain relatively more challenging.

Key Insight

Class-wise evaluation highlights the impact of visual complexity and occlusion on model performance.

ROC Curve Analysis

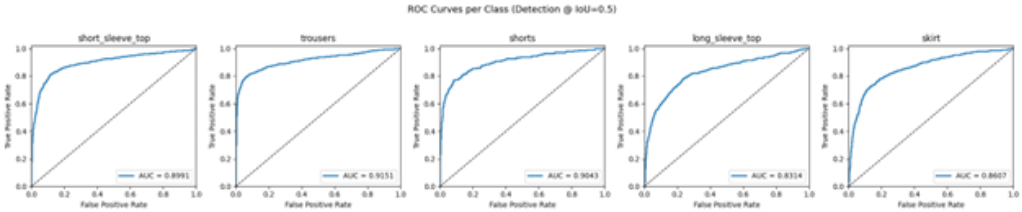

Figure 8: ROC curves for Mask R-CNN across different clothing categories

Observation:

- All classes exhibit high Area Under Curve (AUC) values
- Trousers achieve the highest AUC (0.9151)
- Long sleeve tops show relatively lower performance (0.8314)

Analysis:

- High AUC values indicate strong classification capability
- The model effectively separates object and background regions
- Consistent performance across classes demonstrates robustness

Key Insight

ROC curves confirm that Mask R-CNN achieves reliable classification performance across all categories.

F1 Score vs Confidence Threshold

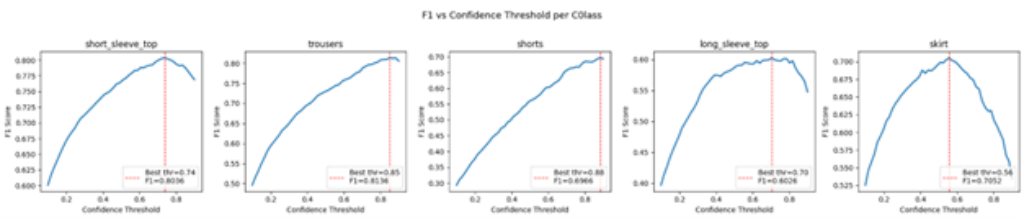

Figure 9: F1-score vs confidence threshold for each class

Observation:

- Each class achieves peak F1 at a specific threshold
- Optimal thresholds vary across classes

- Higher thresholds increase precision but reduce recall

Analysis:

- Trousers achieve the highest F1-score (0.8136)
- Long sleeve tops remain the most challenging class
- Threshold tuning is critical for optimal performance

> **Key Insight**
>
> Class-specific threshold tuning is essential for maximizing detection performance.

Discussion and Insights The experimental evaluation of Mask R-CNN demonstrates its effectiveness in addressing both object detection and instance segmentation tasks. Compared to previous models explored in this project, Mask R-CNN provides a more comprehensive and accurate understanding of complex scenes.

Key Observations

- The model achieves consistently high ROC-AUC scores across all classes
- Optimal threshold tuning significantly improves F1 performance
- Instance-level segmentation enables separation of multiple clothing items

> **Key Insight**
>
> Mask R-CNN effectively combines classification, localization, and segmentation into a unified framework, resulting in superior performance.

Strengths of Mask R-CNN

- **Instance-Level Segmentation:** Generates separate masks for each object, even within the same class
- **High Detection Accuracy:** Strong ROC-AUC values indicate reliable classification performance
- **Precise Localization:** Bounding boxes tightly fit object regions
- **Robust to Complex Scenes:** Handles overlapping and multiple objects effectively

> **Key Insight**
>
> Mask R-CNN addresses the limitations of both U-Net and YOLO by combining their strengths while overcoming their weaknesses.

Limitations Despite its strong performance, some limitations were observed:

- **Higher Computational Cost:** Two-stage architecture results in slower inference compared to YOLO
- **Class-wise Performance Variations:** Some classes (e.g., long sleeve tops) show relatively lower performance
- **Dependency on Threshold Selection:** Performance is sensitive to confidence threshold tuning

Key Insight

The trade-off between accuracy and computational efficiency is a key consideration when using Mask R-CNN.

Comparison with Previous Models

- **Compared to U-Net:** Provides instance-level segmentation instead of merged regions
- **Compared to YOLO:** Achieves higher segmentation precision and better localization
- **Overall:** Combines detection and segmentation in a single unified framework

Key Insight

Mask R-CNN emerges as the most complete solution among all models evaluated in this project.

Final Interpretation

- Mask R-CNN achieves the best balance between detection accuracy and segmentation quality
- It provides detailed and interpretable outputs for complex multi-object scenes
- It outperforms both semantic segmentation and single-stage detection approaches

Key Insight

Mask R-CNN is the most suitable model for this task due to its ability to accurately detect, classify, and segment multiple clothing items simultaneously.

6 Final Comparative Analysis

6.1 Classification Comparison

Model	Training Strategy	Best Val F1
ResNet-50	Transfer Learning	**0.8666**
ResNet-50	Scratch	0.7833
EfficientNet	Transfer Learning	0.8481
EfficientNet	Scratch	0.7141
MobileNetV3	Transfer Learning	0.8233
MobileNetV3	Scratch	0.6685

Table 12: Final comparison of classification models

Per-Class Results: Best Model (ResNet-50 Transfer Learning)

Category	Precision	Recall	F1-Score
Short Sleeve Top	0.9088	0.8825	0.8955
Trousers	**0.9171**	**0.9179**	**0.9175**
Shorts	0.8410	0.8316	0.8363
Long Sleeve Top	0.8332	0.8071	0.8200
Skirt	0.9033	0.8392	0.8701
Macro Average	0.8807	0.8557	0.8679
Micro Average	0.8910	0.8669	0.8788

Table 13: Per-class classification performance for ResNet-50 (Transfer Learning)

Observation:

- ResNet-50 achieves the highest performance with transfer learning
- EfficientNet closely follows, demonstrating strong performance with pretrained weights
- MobileNetV3 provides a good trade-off between efficiency and accuracy

Key Insight:

EfficientNet benefits significantly from transfer learning, achieving performance comparable to ResNet-50 while maintaining a more efficient architecture.

Key Insight

Transfer learning enables deeper architectures like EfficientNet to reach competitive performance levels even with limited data.

ROC Curve Analysis (ResNet-50 Transfer)

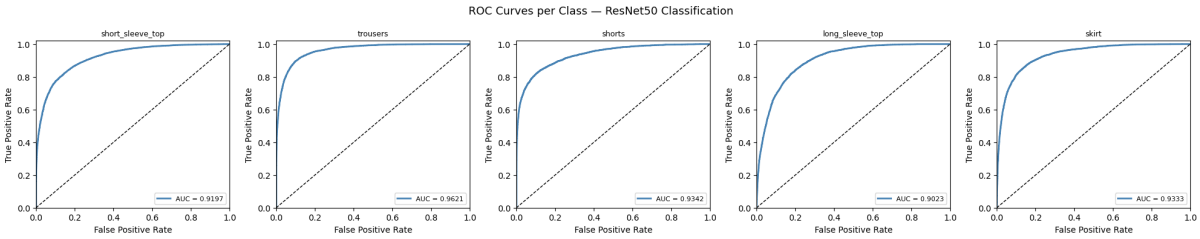

Figure 10: ROC curves for each clothing category using ResNet-50 (Transfer Learning)

Category	AUC
Short Sleeve Top	0.9197
Trousers	**0.9621**
Shorts	0.9342
Long Sleeve Top	0.9023
Skirt	0.9333

Table 14: ROC-AUC scores per class for ResNet-50

Observation:

- All classes achieve high AUC values (>0.90), indicating strong classification performance
- Trousers achieve the highest AUC (0.9621), showing excellent separability
- Long sleeve tops have relatively lower AUC (0.9023), indicating higher confusion with similar classes

Analysis:

- High AUC values indicate that the model effectively distinguishes between classes
- The ROC curves remain close to the top-left corner, demonstrating strong predictive capability
- Consistent performance across classes highlights robust feature extraction by ResNet-50

Key Insight

The high ROC-AUC values across all classes confirm that ResNet-50 with transfer learning provides reliable and well-calibrated classification performance.

6.2 Detection and Segmentation Comparison

Model	Strategy	mIoU	Dice	mAP@0.5	AUC
Mask R-CNN	Transfer	**0.8600**	**0.9218**	0.8049	–
Mask R-CNN	Scratch	0.8517	0.9168	0.6409	–
YOLOv8m	Transfer	0.5238	0.5926	0.6475	**0.9476**
YOLOv8m	Scratch	0.4169	0.4798	0.5278	0.9035
U-Net	Transfer	0.4870	0.5575	–	–
U-Net	Scratch	0.3137	0.3000	–	–

Table 15: Comparison of detection and segmentation models

Performance Analysis

Observation:

- Mask R-CNN achieves the highest segmentation performance with mIoU of 0.86 and Dice score of 0.92
- YOLO achieves strong detection performance with high AUC
- U-Net performs well in semantic segmentation but lacks instance-level understanding

Key Insight:

Mask R-CNN significantly outperforms all models in segmentation accuracy, while also providing instance-level predictions that are not captured by U-Net or YOLO.

Key Insight

Mask R-CNN is the most comprehensive model as it combines detection, classification, and precise instance-level segmentation.

6.3 Overall Insights

Key Observations:

- Transfer learning consistently improves performance across all models and tasks
- Mask R-CNN achieves the highest segmentation performance with mIoU of 0.86 and Dice score of 0.92
- YOLO demonstrates strong detection capability with high AUC and efficient inference
- U-Net performs well in semantic segmentation but lacks instance-level understanding

Task-wise Model Suitability:

- **Classification:** ResNet-50 provides the best performance due to strong feature extraction
- **Detection:** YOLO is efficient and performs well for real-time detection tasks
- **Segmentation:** Mask R-CNN significantly outperforms other models in segmentation accuracy

Important Insight:

While YOLO achieves strong detection metrics and U-Net provides reasonable segmentation performance, Mask R-CNN delivers the most complete solution by combining accurate detection with precise instance-level segmentation.

Key Insight

Mask R-CNN is the most suitable model for real-world applications involving multiple overlapping clothing items, as it provides detailed and interpretable outputs at both object and pixel levels.